systems. For other operating systems you'll need to call out to other commands in order to achieve the same results. That said, the techniques demonstrated here all still apply, so you can still follow along.

Write the configuration as a "plain-old object" and expose it as a module. A server.coffee file that uses a configuration loaded from a local `config` module appears in the following listing.

Listing 12.17. The blog application (server.coffee)

```
http = require 'http'
{load} = require './load'
{Blog} = require './controllers'

load './content'

config = require('./config')[process.env.NODE_ENV]

server = new http.Server()
server.listen config.port, config.host

blog = new Blog server
```

Require the config file and get the export that matches the current environment.

The local `config` module appears in the next listing. There's a configuration for the development and production environments. Any other environments you have (such as testing or staging environments) should also be defined in this file.

Listing 12.18. The blog application configuration file (config.coffee)

```
config =
  development:
    host: 'localhost'
    port: '8080'
  production:
    host: 'agtronsblog.com'
    port: '80'

for key, value of config
  exports[key] = value
```

Your application is now ready to run in a specific environment with a particular configuration. So how does the environment variable get set?

Defining the environment